

KVMillésime - Vendanges tardives

Ce challenge s'intéresse aux problèmes d'ingénierie que représente la virtualisation. Exemple simple : les *snapshots*.

Gist

Le device `qemu` n'est pas correctement sauvegardé. Le compteur de base n'est pas inclus dans la sauvegarde, cassant l'attaque triviale. Mais comme l'état du PRNG est sauvegardé, on peut quand même exploiter les snapshots.

Tour d'horizon

Fonctionnement de la machine QEMU

En lisant `qemu-pci-device.c`, on apprend le fonctionnement global du device. Le code est presque entièrement du boiler-plate QEMU, sauf pour les sections d'écriture et lecture. On a un générateur aléatoire à base de LFSR, et une constante incrémentée à chaque appel en lecture ou écriture.

```
static void pci_rng_ascending_write(void *opaque, hwaddr addr, uint64_t val, unsigned size)
{
    PCIRngAscendingState *d = opaque;
    if (addr == 0x08 && size == 8) {
        uint64_t r = (xorshift128plus(d->lfsr_state) % 127) + 1;
        d->n_counter += r;
        d->last_guess_ok = (val == d->n_counter);
    } else {
        qemu_log_mask(LOG_GUEST_ERROR, "pci-rng-ascending: write at offset 0x%"
HWADDR_PRIx "\n", addr);
    }
}
```

A priori, on veut essayer de deviner `n_counter`.

Étant donné que le service auquel nous accédons nous laisse prendre et charger un snapshot de la VM, on se doute que le problème vient de la sauvegarde.

Trivial du coup ? On sauvegarde, on GET, on restaure, on SET cette valeur ?

La sauvegarde trop légère

Le code du device (`qemu-pci-device.c`) définit son VMState par

```
static const VMStateDescription vmstate_pci_rng_ascending = {
    .name = "pci-rng-ascending",
    .version_id = 1,
    .minimum_version_id = 1,
    .fields = (const VMStateField[]) {
        VMSTATE_PCI_DEVICE(parent_obj, PCIRngAscendingState),
        VMSTATE_UINT64_ARRAY(lfsr_state, PCIRngAscendingState, 2),
        VMSTATE_UINT32(last_guess_ok, PCIRngAscendingState),
        VMSTATE_END_OF_LIST()
    }
};
```

En parcourant la documentation ([1] ou en suivant son instinct), on apprend que cet état sera *la seule chose* sauvegardée puis restaurée en cas de snapshot. Ici, `lfsr_state` est bien sauvegardé, mais pas

`n_counter`. En cas de snapshot, `n_counter` continuera d'exister comme si la recharge n'avait jamais eu lieu.

Donc la méthode triviale ci-dessus ne marche pas. Voici ce qu'il se passe, en notant N la valeur initiale de `n_counter`, et k la valeur aléatoire.

```
SAVE
GET
-> donne  $N+k$ 
RESTORE
GET
-> donne  $N+k+k$ , car n_counter n'a pas été restauré
```

Mais comme k ne change pas, on peut facilement déduire le coup suivant.

```
SAVE
GET ->  $N+k$ 
RESTORE
GET ->  $N+2*k$ 
Dédire  $k$ 
RESTORE
GUESS  $N+3*k$ 
```

Voir `solve.py` pour une implémentation.

Note: On pouvait aussi casser le LFSR. C'est plus marrant, mais plus difficile.

Bibliography

- [1] "Migration Framework." [Online]. Available: <https://www.qemu.org/docs/master/devel/migration/main.html#general-advice-for-device-developers>